

Aplicación en la Bioinformática

Laboratorio No.2 de Computación Cuántica.

Jose Andres Henao Alzate

*Ing. Electrónica y
Telecomunicaciones*

Universidad de Antioquia

Medellín, Colombia

andres.henao9@udea.edu.co

Santiago Giraldo Tabares

*Ing. Electrónica y
Telecomunicaciones*

Universidad de Antioquia

Medellín, Colombia

santiago.giraldot@udea.edu.co

Jakob Kuhn

*Ing. Electrónica y
Telecomunicaciones*

Universidad de Antioquia

Medellín, Colombia

kuhn.jakob@mail.de

Abstract—El presente informe aborda la comparación de secuencias genéticas mediante técnicas de computación cuántica, utilizando el framework Qiskit. Se implementa un procedimiento de codificación de cadenas genéticas en circuitos cuánticos, los cuales representan los estados cuánticos de microorganismos y especies biológicas. A partir del uso de circuitos inversos y composiciones de compuertas, se estudia la fidelidad cuántica como medida de similitud entre distintas secuencias genéticas. Esta metodología aprovecha propiedades fundamentales de la computación cuántica, como la reversibilidad y la superposición, para determinar el grado de similitud entre cadenas, utilizando observables definidos con SparsePauliOp y estimadores de estado cuántico (StatevectorEstimator). Finalmente, se presentan comparaciones entre secuencias de diferentes organismos, así como un análisis detallado del comportamiento de la fidelidad cuántica ante cambios mínimos en las cadenas evaluadas.

Index Terms—Computación, Genética, RNA, ADN, microorganismos.

I. INTRODUCCIÓN

En bioinformática, una de las tareas más relevantes es la comparación de secuencias genéticas provenientes del ADN o ARN de diferentes especies. Estas secuencias, representadas como cadenas de caracteres en un alfabeto genético compuesto por los nucleótidos “A”, “C”, “G” y “T”, contienen información esencial sobre la identidad biológica de un organismo. Tradicionalmente, estas comparaciones se realizan con algoritmos clásicos que buscan patrones o alineaciones, sin embargo, con el auge de la computación cuántica, han surgido nuevas formas de representar, manipular y comparar estas cadenas de manera más eficiente.

Este informe explora una técnica cuántica basada en codificación binaria de secuencias y aplicación de compuertas cuánticas para medir la similitud entre estados cuánticos que representan cadenas genéticas. En particular, se estudia el concepto de fidelidad cuántica, el cual permite cuantificar la similitud entre dos estados mediante observables definidos sobre los circuitos. Se parte de un tutorial base en Qiskit que utiliza secuencias representadas en ASCII y se adapta para trabajar con cadenas genéticas reales, codificadas en un alfabeto de cuatro símbolos.

El código fuente analizado implementa el proceso de codificación de una cadena a un circuito cuántico, su inversión para posterior comparación, y finalmente la composición de

circuitos para evaluar qué tan similares son dos secuencias. A través de simulaciones con miles de mediciones (shots), se calcula el porcentaje de coincidencias absolutas entre dos estados, lo cual permite establecer una métrica de fidelidad. Esta técnica se valida comparando una cadena consigo misma, luego con una versión modificada, y finalmente entre secuencias de distintos organismos. Esta comparación cuántica abre una nueva perspectiva para el análisis de información genética en sistemas biológicos complejos.

II. EXPLICACIÓN DEL CÓDIGO ENTREGADO

1.1. ANÁLISIS DEL TUTORIAL *Comparing Strings with Quantum Superposition*

El tutorial, actualizado a Qiskit 2.0.0 por el Prof. Gustavo Patiño y originalmente publicado en el repositorio de tutoriales de Qiskit, propone un método cuántico para medir el grado de similitud entre cadenas genéticas de distintas especies biológicas. El tutorial propone su método con base a dos principios clave:

- 1) Rápida codificación de una cadena genética en términos de un estado cuántico (una superposición cuántica).
- 2) Rápida comparación de dos cadenas mediante el principio de la reversibilidad de los operandos cuánticos.

Inicialmente, y mientras se da un preámbulo sobre cómo se realiza la codificación genética mediante caracteres ‘M’ y ‘-’, se crean tres variables, llamadas YEAST, PROTOZOAN y BACTERIAL, las cuales almacenan las cadenas de caracteres que serán objeto de comparación. Posteriormente:

- Se explica qué **componentes y librerías de Qiskit** han de **importarse** para esta labor. Así mismo, se deja explícita la configuración del **simulador local (AerSimulator)** con el que se llevarán a cabo las operaciones y la elaboración de circuitos cuánticos.
- se propone una **función `encode_bitstring(...)`** para codificar las cadenas genéticas (cadenas de STARTs) en circuitos cuánticos equivalentes.

- Posteriormente, se emplea dicha función para **crear los circuitos cuánticos correspondientes** a las tres cadenas de STARTs de YEAST, PROTOZOAN y BACTERIAL (circuitos U de preparación). Tras esto, se emplea la misma función para **obtener los circuitos invertidos** (U^{-1}) para las tres cadenas.
- Para comparar, se **compone U de la cadena “clave” con el inverso $V^\dagger$** de otra, se simula y se mide la frecuencia con que el sistema retorna al estado $|0\rangle^{\otimes n}$. Esa frecuencia—sobre el total de *shots*—es el *similarity score*, que **cuantifica la similitud cuántica** entre las dos cadenas.

1.2. REPRESENTACIÓN CUÁNTICA DE LAS SECUENCIAS

Para codificar una única cadena de 64 caracteres se usa un *registro cuántico* de 7 qubits ($2^7 = 128$ estados base para representar):

- Seis qubits codifican la posición $i \in \{0, \dots, 63\}$.
- Un qubit codifica el valor en esa posición: $|0\rangle$ si es ‘-’, $|1\rangle$ si es ‘M’.

A cada una de las 64 posiciones con valor ‘-’ o ‘M’ se le asigna una *amplitud*

$$\alpha = \frac{1}{\sqrt{64}} = \frac{1}{8},$$

de modo que la *probabilidad* de cada término sea $|\alpha|^2 = 1/64$ y, en conjunto,

$$\sum_{i=0}^{63} |\alpha|^2 = 64 \times \frac{1}{64} = 1.$$

Esta normalización garantiza que el vector de estado sea unitario. Los índices no correspondientes a la cadena tienen amplitud cero. Así, el estado resultante es

$$|\psi\rangle = \sum_{i=0}^{63} \alpha |i\rangle |v_i\rangle, \quad v_i = \begin{cases} 0, & \text{si ‘-’,} \\ 1, & \text{si ‘M’}. \end{cases}$$

En Qiskit se implementa con `Initialize(desired_vector)`.

Ejemplo: Para `TEST = \-M--` (4 caracteres, 3 qubits),

$$|\phi\rangle = \frac{1}{2}(|00\rangle|0\rangle + |01\rangle|1\rangle + |10\rangle|0\rangle + |11\rangle|0\rangle),$$

dejando las demás combinaciones con amplitud cero...

Con esto, nuestro **espacio computacional de 7 qubits, o 128 posiciones**, tendría **64 de estas con un valor de probabilidad asociado de 1/64 o amplitudes cuánticas de 1/8**. Todos aquellos 64 elementos que componen la cadena genética se **superpondrán (equiprobablemente en este caso)**, generando un **vector de estado único** cuya ubicación en el espacio cuántico estará dictada por qué **combinaciones tienen una amplitud cuántica diferente de cero**. Con esto,

dos cadenas de caracteres diferentes tendrán vectores de estado diferentes, pues estos tendrán combinaciones no nulas diferentes entre sí. La **superposición de estas combinaciones caracteriza a una y solo una cadena en cuestión**. Este principio es útil, pues este **vector de estado único será llevado a un circuito cuántico equivalente, también único**.

1.3. FUNCIÓN `encode_bitstring(...)`

La función `encode_bitstring()`, como bien lo indica su nombre, tiene como fin **codificar las cadenas de caracteres correspondientes a las cadenas de START** de los genomas de interés, en términos de **puertas cuánticas**. O dicho de manera simple, **convierte una cadena de caracteres en un correspondiente circuito cuántico**. Esto permite que a cada cadena genética le corresponda, al final, un **estado cuántico único y característico**.

La función permite generar tanto los circuitos directos como sus circuitos inversos. Recibe como argumentos:

- `bitstring`: Cadena de caracteres a partir de la cual se creará el circuito cuántico.
- `qr`: Registro cuántico con el que se creará el circuito cuántico.
- `cr`: Registro clásico utilizado para la medición del circuito (solo cuando `inverse=True`).
- `inverse`: Parámetro booleano que le indica a la función si debe crear el circuito directo o su inverso.

La función retornará `qc`, el circuito cuántico, invertido o no, según lo solicitado mediante el parámetro `inverse`.

A continuación, se presenta un análisis más detallado de la función:

```
1 def encode_bitstring(bitstring, qr, cr,
   ↪ inverse=False):
```

Sintaxis y orden de los parámetros para la definición de la función

```
1     n = math.ceil(math.log2(len(bitstring))) + 1
   ↪     #number of qubits
2     assert n > 2, "the length of bitstring must
   ↪     be at least 2"
3
4     print("Here, a quantum circuit consisting
   ↪     of", n, "qubits will be required. \n")
5
6     qc = QuantumCircuit(qr, cr)
```

Este bloque contiene las instrucciones para la creación del circuito cuántico. Lo principal es **definir el tamaño del registro**, es decir, cuántos qubits se requieren para construir los registros cuántico y clásico. Para ello, se define `n` como el número total de qubits necesarios (y también la cantidad de bits clásicos en el registro clásico), utilizando la fórmula:

$$n = \lceil \log_2(\text{longitud del bitstring}) \rceil + 1$$

donde:

- $\log_2(\text{longitud-bitstring})$ determina el número de qubits necesarios para representar el **índice** de cada carácter en la cadena de entrada.
- La **función techo** ($\lceil \cdot \rceil$) se utiliza porque la operación $\log_2$ por sí sola puede entregar un número no entero. Dado que no se pueden tener 3.5 qubits, 12.89 qubits, etc., se necesita redondear hacia arriba para asegurar que el número de qubits sea **entero y suficiente** para representar todas las posiciones (índices) del bitstring.
- El **+1 adicional** implica que se agrega un qubit extra para almacenar el **valor** de cada carácter en dicha posición. En este caso, basta con un solo qubit adicional porque solo existen dos valores posibles ('M' y '-'), que se codifican como $|1\rangle$ y $|0\rangle$ respectivamente.
- Desde esta lógica, se infiere que si se quisiera representar cadenas en términos de nucleótidos (A, G, C, T), sería necesario modificar esta fórmula, sustituyendo el +1 por +2, para codificar cada valor como un par de bits: A $\rightarrow$ 00, G $\rightarrow$ 01, C $\rightarrow$ 10, T $\rightarrow$ 11. En general, para un conjunto de m caracteres distintos, se requeriría añadir $\log_2(m)$ qubits adicionales.

Nota: En la línea 2, `assert n > 2` valida que el número total de qubits sea mayor a 2, pues se sobre-entiende que lo mínimo válido es una cadena de 1 carácter, con dos posibles valores.

Se imprime por consola la cantidad total de qubits que tendrá el circuito cuántico generado, a modo informativo.

Finalmente, se crea el circuito cuántico principal `qc` mediante el método `QuantumCircuit` de Qiskit, utilizando los registros cuántico (`qr`) y clásico (`cr`) que fueron pasados como argumentos.

```
1  #the probability amplitude of the desired
   ↪ state
2  desired_vector = np.array([ 0.0 for i in
   ↪ range(2**n) ]) #initialize to zero
```

En esta línea de código se crea un **vector unidimensional de tamaño 2^n** , con capacidad para representar **todas las posibles combinaciones de clave-valor**, según la longitud del bitstring y la cantidad de posibles valores por carácter.

Por lo tanto, este vector funcionará como una **representación única del estado cuántico asociado a cada cadena**, y será posteriormente convertido en un circuito cuántico utilizando la instrucción `Initialize()` de Qiskit.

El vector se inicializa completamente en ceros. Es importante notar que se llena con 0.0, lo cual es una forma implícita de definirlo como un **arreglo de flotantes**. Esto es necesario porque los coeficientes cuánticos que se asignarán después (es decir, las **amplitudes asociadas a su probabilidad**) son números reales.

```
1  qc_init = QuantumCircuit(n)
2  inverse_qc_init = QuantumCircuit(n)
```

```
3
4  amplitud = np.sqrt(1.0/2**(n-1))
```

En las líneas 1 y 2 del bloque de código anterior, se crean las **variables auxiliares** —o mejor dicho, **circuitos cuánticos auxiliares**— que contendrán, respectivamente, los circuitos U y $U^\dagger$ (este último si `inverse=True`), ambos con la misma cantidad de qubits necesaria para representar el estado cuántico deseado.

En la línea 4 del código, se define el valor de la **amplitud cuántica**, el cual será simplemente la raíz cuadrada de la probabilidad:

$$\text{amplitud} = \sqrt{\frac{1}{2^{n-1}}}$$

Esto es posible debido a que, en este caso, las combinaciones clave-valor que se desean codificar son **equiprobables**, por lo que basta con una única variable `amplitud` para asignar el mismo valor a cada estado válido del vector.

```
1  for i, b in enumerate(bitstring):
2      pos = i * 2
3      if b == "1" or b == "M":
4          pos += 1
5      desired_vector[pos] = amplitud # What
   ↪ has just happened here?
```

En el bloque de código anterior, se implementa un **bucle for** que recorre la cadena de entrada (`bitstring`) y va **rellenando el arreglo desired_vector**, previamente inicializado con ceros (0.0), con el valor de la amplitud cuántica calculada. La lógica del bucle es la siguiente:

- 1) El bucle recorre el `bitstring` carácter por carácter (por ejemplo, si la cadena tiene 64 caracteres, se realizarán 64 iteraciones).
- 2) En cada iteración, se define una variable `pos` como el doble del índice `i` actual del bucle. Es decir, `pos = 2 * i`.
- 3) Luego, se evalúa qué carácter hay en la posición `i` del `bitstring`. Si el carácter es 'M' o '1', entonces se suma 1 a `pos`. Si no, `pos` permanece igual.
- 4) Finalmente, al arreglo `desired_vector` en la posición `pos` se le asigna el valor de `amplitud`.

Lo que se hace, entonces, es **asignar amplitud a ciertas posiciones del vector desired_vector** dependiendo del carácter en esa posición de la cadena. En concreto:

- `amplitud` se almacena en **posiciones pares** si el carácter es '-'
- `amplitud` se almacena en **posiciones impares** si el carácter es 'M'

Todos los valores de `desired_vector` que no sean modificados por el bucle (es decir, aquellas combinaciones índice-valor que no están presentes en la cadena) permanecen con valor 0.0.

En total, para este caso particular:

- **64 combinaciones índice-valor** tienen amplitud 1/8

- **64 combinaciones restantes** tienen amplitud 0

Estas 64 combinaciones con amplitud no nula son las únicas que forman parte del estado cuántico codificado. La lógica de llenado del arreglo se ejemplifica con la siguiente imagen:

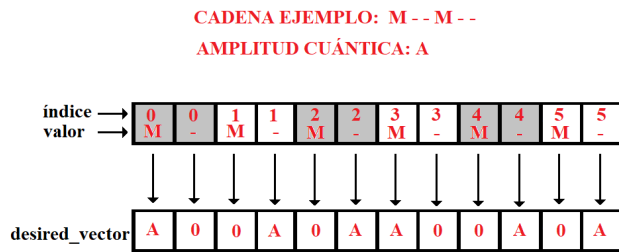


Fig. 1. Ejemplo de construcción de desired_vector

```
1 init = Initialize(desired_vector)
```

En la línea de código correspondiente, se emplea la instrucción `Initialize` de Qiskit, la cual, cuando se aplica sobre qubits en estado $|0\rangle^{\otimes n}$, los transforma en el estado especificado por `desired_vector`, que representa la **superposición cuántica codificada del bitstring**.

En resumidas cuentas, lo que se hace es **almacenar en la variable `init` la instrucción que, al aplicarse, prepara el estado cuántico deseado a partir del vector de amplitudes `desired_vector`**.

```
1 if not inverse:
2     qc_init.append(init, qc_init.qubits)
3     qc.append(qc_init, qr)
4     qc.barrier(qr)
```

Este bloque de código se ejecuta si el parámetro `inverse` es **False**, es decir, si se desea construir el **circuito directo** U que prepara el estado cuántico deseado.

- `qc_init.append(init, qc_init.qubits)`
En esta línea se agrega la instrucción `init` (que contiene la operación `Initialize`) al circuito auxiliar `qc_init`, el cual tiene los n qubits requeridos para representar el estado deseado. Esta instrucción implementa el estado cuántico a partir del vector de amplitudes generado anteriormente.
- `qc.append(qc_init, qr)`
Aquí se inserta el subcircuito `qc_init` (ya con la inicialización cargada) dentro del circuito principal `qc`, usando el registro cuántico externo `qr` como destino. De esta manera, el circuito final hereda el estado $|\psi\rangle$ preparado por `qc_init`.
- `qc.barrier(qr)`
Finalmente, se añade una barrera sobre los qubits del registro `qr`. Esta barrera no afecta la computación, pero actúa como una separación simbólica entre las etapas del

circuito, facilitando su interpretación. También se utiliza para evitar que, en caso de "transpilar" el circuito, las compuertas no se reacomoden cruzando las barreras.

```
1 else:
2     uncompute =
3         init.gates_to_uncompute().decompose()
4         inverse_qc_init.append(uncompute,
5                                 inverse_qc_init.qubits)
6     qc.append(inverse_qc_init, qr)
7     qc.barrier(qr)
8     for i in range(n):
9         qc.measure(qr[i], cr[i])
```

Por otro lado, si se desea obtener el circuito invertido **circuito inverso** $U^\dagger$ (es decir, si `inverse = True`) se ejecuta el anterior bloque.

- `uncompute = init.gates_to_uncompute().decompose()`

Esta línea genera el circuito inverso de la operación `init`. Internamente, Qiskit invierte la secuencia de puertas cuánticas utilizadas por `Initialize` para producir un circuito que revierta la preparación del estado. La función `.decompose()` descompone esta instrucción en puertas básicas visibles y manipulables.

Ahora, similar a como se hizo para el caso en que `Inverse=False`,

- `inverse_qc_init.append(uncompute, inverse_qc_init.qubits)`

Se inserta la instrucción inversa en el circuito auxiliar `inverse_qc_init`, que está diseñado para ejecutar $U^\dagger$ sobre los mismos n qubits.

- `qc.append(inverse_qc_init, qr)`

Se inserta el circuito inverso `inverse_qc_init` dentro del circuito principal `qc`, utilizando el registro cuántico `qr`. Esto permite aplicar la operación inversa directamente sobre el estado actual de los qubits.

- `qc.barrier(qr)`

Se agrega una barrera al circuito

- `for i in range(n): qc.measure(qr[i], cr[i])`

Finalmente, se mide cada uno de los n qubits del registro cuántico `qr` y se almacenan los resultados en los bits correspondientes del registro clásico `cr`. **Esto colapsa el estado cuántico y permite observar el resultado de aplicar $U^\dagger$ sobre un estado preparado previamente.**

Ahora, teniendo en cuenta el funcionamiento de la función `encode_bitstring(...)` y cómo esta permite **convertir una cadena de caracteres en un estado cuántico**, así como generar su **circuito cuántico inverso**, se procede a **aprovechar la reversibilidad de las operaciones cuánticas**

para analizar la similitud entre cadenas.

1.4. ¿POR QUÉ INVERTIR LOS CIRCUITOS CUÁNTICOS?

Como ya se ha mencionado, una característica diferenciadora de las compuertas cuánticas con respecto a las clásicas, es que es posible revertirlas, sin perder información sobre la entrada. Es decir, aplicado un operando cuántico a un estado base, si al estado cuántico resultante se le aplica el inverso del operando cuántico empleado, se obtendrá como resultado la entrada inicial. O dicho de otro modo, un circuito cuántico concatenado directamente con su inverso, tendrá a su entrada y salida el mismo estado cuántico inalterado.

Ahora bien, ¿cómo puede servir esto para comparar cadenas de caracteres?. Pues, supóngase un escenario ficticio, en el que **Jose** puede expresarse en terminos de un estado cuántico U , y que **Jakob** puede expresarse en terminos de un estado cuántico S . Ahora, supóngase que se compone el circuito $U \cdot S^\dagger$. Si a ese circuito compuesto, se le ingresa un **estado base** (por ejemplo, $|0000000\rangle$) y el **estado cuántico a la salida es igual al de la entrada** ($|0000000\rangle$), se puede concluir que $U = S$. Es decir, las matrices que caracterizan a **Jose** y a **Jakob** son las mismas, con lo que ¡Jose y Jakob son iguales!

Ahora, volviendo al análisis de las cadenas de STARTs en los genomas de YEAST, PROTOZOAN y BACTERIAL, se ha de **escoger una de las tres cadenas** como referencia para comparar con las otras dos. A la cadena elegida se le construye el **circuito directo** U , mientras que a las otras dos se les generan los **circuitos inversos** $V^\dagger$ y $W^\dagger$.

Luego, se **componen los circuitos** $U \cdot V^\dagger$ y $U \cdot W^\dagger$ para cada comparación, con el fin de analizar la similitud entre las cadenas.

Ahora, si se ejecuta el circuito compuesto, con una entrada ($|0000000\rangle$), a la salida se obtendrá ($|0000000\rangle$) tras un numero suficiente de tiradas, siempre que, de los dos circuitos que componen el circuito compuesto, uno sea el inverso del otro. Pero si no se da ese caso, las discrepancias entre uno y otro circuito introducirán variaciones en la salida.

De todas las veces que se ejecute el circuito compuesto, se contabilizan aquellas que hayan mantenido el estado base inalterado (se contabilizan la cantidad de ($|0000000\rangle$) obtenidos a la salida). Con esto, se calcula el puntaje de similitud como una tasa de ($|0000000\rangle$) con respecto al total de experimentos.

Si la totalidad de experimentos resultan en ($|0000000\rangle$), quiere decir que los circuitos concatenados son el uno, el inverso del otro. o dicho de otro modo, las cadenas a comparar son exactamente iguales.

como ejemplo, suponga las siguientes cadenas genéticas:

- Cadena A: - M - - - M - M M
- Cadena B: - M - - - M - M M

Construcción del circuito U : El circuito U es aquel que transforma el estado base $|00000\rangle$ en el estado codificado:

$$U|00000\rangle = |\psi_A\rangle$$

Construcción del circuito inverso $U^\dagger$: El circuito inverso $U^\dagger$ es simplemente la secuencia de puertas que deshace la preparación del estado:

$$U^\dagger|\psi_A\rangle = |00000\rangle$$

Comparación cuántica entre cadenas: Suponga que se desea comparar la cadena A con la cadena B, generando los circuitos:

- U : circuito que codifica la cadena A
- V : circuito que codifica la cadena B

Cada uno de estos circuitos prepara un estado cuántico asociado a su respectiva cadena. En particular:

$$U|00000\rangle = |\psi_A\rangle$$

$$V|00000\rangle = |\psi_B\rangle$$

Luego, se compone el circuito:

$$C = V^\dagger \cdot U$$

Y se aplica sobre el estado base:

$$C|00000\rangle = V^\dagger U|00000\rangle = V^\dagger|\psi_A\rangle$$

Si los estados cuánticos generados por ambas cadenas son iguales, es decir, si $|\psi_A\rangle = |\psi_B\rangle$, entonces:

$$V^\dagger|\psi_A\rangle = V^\dagger|\psi_B\rangle = |00000\rangle$$

Es decir, si ambas cadenas codifican el mismo estado cuántico, la composición $V^\dagger U$ deja el sistema sin cambios. En ese caso, **medir el estado** $|00000\rangle$ al final del circuito compuesto indicaría que ambas cadenas son idénticas.

Este proceso se repite muchas veces (8192 *shots*) y se mide qué tan frecuentemente se obtiene el estado base. El **similarity score** se calcula como:

$$\text{Similarity Score} = \frac{\text{número de veces que aparece } |00000\rangle}{8192}$$

Un valor cercano a 1 indica alta similitud entre cadenas. Un valor bajo indica diferencia estructural entre los vectores cuánticos que representan ambas cadenas de STARTs.

III. ANÁLISIS DE RESULTADOS

El código inicialmente propone tres cadenas de caracteres, entre las cuales resulta evidente a simple vista que dos de ellas (PROTOZOAN y BACTERIAL) son muy similares entre sí, y suficientemente diferentes de una tercera (YEAST):

Por lo que es de esperarse que, tras la ejecución del *Script*, el *similarity score* entre BACTERIAL y YEAST sea significativamente menor que el que haya entre BACTERIAL y PROTOZOAN.

Tras ejecutar el circuito, el resultado obtenido es el siguiente:

[IMAGEN 4]

Nótese que se cumple con lo provisto inicialmente. Las cadenas de START de BACTERIAL y PROTOZOAN son bastante parecidas, mientras que el porcentaje obtenido indica que BACTERIAL y YEAST son menos similares entre sí. No obstante, se esperaba una mayor diferencia entre estas dos últimas. Teniendo en cuenta que cada ejecución (*shot*) del circuito puede colapsar en el resultado correcto o no, y que la naturaleza de la computación cuántica es intrínsecamente probabilística, es posible que, para aumentar la precisión de los **puntajes de similitud**, sea necesario aumentar el número de *shots* del experimento.

2. FIDELIDAD CUÁNTICA

2.1 Análisis Detallado

¿Qué componentes intervienen en la medición de la fidelidad cuántica?: Los componentes principales que intervienen son:

Componente	Función
$ \psi\rangle$ (estado cuántico preparado)	Estado que hemos construido mediante una operación cuántica U .
$ \phi\rangle$ (estado objetivo)	Estado de referencia o de comparación después de aplicar $U^\dagger$.
U (operador cuántico)	Prepara el estado $ \psi\rangle$ desde el estado base $ 0\rangle^{\otimes n}$.
$U^\dagger$ (operador inverso)	Inverso de U , que deshace la preparación del estado cuántico.
Π_0 (observable)	Operador proyector sobre $ 0\rangle^{\otimes n}$, usado para medir la fidelidad.
Medición	Verificamos si el sistema regresa al estado base $ 0\rangle^{\otimes n}$.

¿Cómo se realiza el cálculo de la fidelidad cuántica?: El cálculo sigue los siguientes pasos:

- 1) **Preparación:** Se parte del estado inicial $|0\rangle^{\otimes n}$.

- 2) **Aplicar U :** Se aplica una operación cuántica U para crear el estado preparado:

$$|\psi\rangle = U|0\rangle^{\otimes n}$$

- 3) **Aplicar $U^\dagger$:** Se aplica la operación inversa $U^\dagger$ sobre $|\psi\rangle$, obteniendo:

$$|\phi\rangle = U^\dagger|\psi\rangle$$

- 4) **Medición mediante observable Π_0 :** Se mide la expectativa del observable $\Pi_0 = |0\rangle\langle 0|^{\otimes n}$ en el estado $|\phi\rangle$:

$$F = \langle\phi|\Pi_0|\phi\rangle$$

- 5) **Interpretación:**

- Si $F = 1$, el estado ha vuelto exactamente a $|0\rangle^{\otimes n}$ (fidelidad perfecta).
- Si $F < 1$, hay una diferencia entre los estados (fidelidad parcial).

¿Cuáles son los operadores cuánticos requeridos?: Los operadores necesarios son:

- U : Operador unitario que prepara el estado deseado a partir de $|0\rangle^{\otimes n}$.
- $U^\dagger$: Inverso de U , también llamado operador adjunto (transposición conjugada).
- Π_0 : Observable proyector sobre el estado base $|0\rangle^{\otimes n}$.

Resumen Conceptual

El proceso completo puede entenderse así:

- 1) Partimos del estado base $|0\rangle^{\otimes n}$.
- 2) Aplicamos U para preparar un estado cuántico $|\psi\rangle$.
- 3) Aplicamos $U^\dagger$ para intentar regresar al estado inicial.
- 4) Medimos si hemos vuelto a $|0\rangle^{\otimes n}$ mediante el observable Π_0 .

Interpretación matemática:

$$|\psi\rangle = U|0\rangle^{\otimes n} \longrightarrow |\phi\rangle = U^\dagger|\psi\rangle \longrightarrow F = \langle\phi|\Pi_0|\phi\rangle$$

Visualización conceptual:

- U : Movimiento hacia un estado complicado.
- $U^\dagger$: Deshacer el movimiento.
- Medición con Π_0 : ¿Estamos de nuevo en el punto de partida?

Conclusión: Medimos cuánta "reversibilidad cuántica" existe: si el proceso de ir y volver es perfecto, la fidelidad es máxima.

2.2 Estudio del Código de Fidelidad Cuántica en Qiskit

El código implementa la medición de fidelidad cuántica en dos ejemplos usando Qiskit. Se utilizan los siguientes componentes:

- **QuantumCircuit:** para construir circuitos cuánticos,

- **SparsePauliOp**: para definir observables basados en matrices de Pauli,
- **StatevectorEstimator**: para calcular valores esperados sobre estados finales,
- **Pass Manager**: para optimizar los circuitos antes de su simulación.

Flujo de Trabajo

- 1) Crear un circuito cuántico que define el estado a evaluar.
- 2) Definir un observable que proyecta sobre el estado base deseado ($|0\rangle$ o $|00\rangle$).
- 3) Optimizar el circuito usando el Pass Manager.
- 4) Aplicar el Estimador de Statevector para calcular el valor esperado de la observable.
- 5) Interpretar el valor esperado como la fidelidad cuántica.

Explicación Paso a Paso

Ejemplo 1: Fidelidad entre $|+\rangle$ y $|0\rangle$ (1 Qubit):

- Se crea un circuito con un solo qubit y se aplica una compuerta Hadamard:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

- Observable utilizada:

$$\Pi_0 = \frac{1}{2}(I + Z)$$

Esto equivale al proyector sobre el estado $|0\rangle$:

$$|0\rangle\langle 0| = \frac{I + Z}{2}$$

- Se optimiza el circuito y se calcula la fidelidad midiendo el valor esperado de Π_0 .
- Resultado: aproximadamente 0.5, ya que $|+\rangle$ y $|0\rangle$ tienen un solapamiento de 50%.

Ejemplo 2: Fidelidad perfecta tras aplicar U y luego $U^\dagger$ (2 Qubits):

- Se crea un circuito con dos qubits.
- Se aplican compuertas Hadamard dos veces a cada qubit:

$$H(H(|0\rangle)) = |0\rangle$$

Así que $U = U^\dagger = H^{\otimes 2}$ y el sistema vuelve al estado base $|00\rangle$.

- El observable utilizado es:

$$\Pi_0 = \frac{1}{4}(II + IZ + ZI + ZZ)$$

¿Por qué las Observables son Diferentes?

La forma del observable Π_0 depende del número de qubits y del estado específico que queremos medir.

Caso de 1 Qubit: Queremos medir si el qubit está en $|0\rangle$. Sabemos que:

$$|0\rangle\langle 0| = \frac{I + Z}{2}$$

por lo tanto, el observable correcto es:

$$\Pi_0 = \frac{1}{2}(I + Z)$$

Caso de 2 Qubits: Queremos medir si ambos qubits están en el estado $|00\rangle$.

Primero, recordamos que para un solo qubit:

$$|0\rangle\langle 0| = \frac{I + Z}{2}$$

Así que para dos qubits, el operador proyector es el producto tensorial:

$$|00\rangle\langle 00| = \left(\frac{I + Z}{2}\right) \otimes \left(\frac{I + Z}{2}\right)$$

Expandiendo el producto tensorial:

$$\left(\frac{I + Z}{2}\right) \otimes \left(\frac{I + Z}{2}\right) = \frac{1}{4}(II + IZ + ZI + ZZ)$$

donde:

- II mide la identidad sobre ambos qubits,
- IZ mide Pauli-Z sobre el segundo qubit,
- ZI mide Pauli-Z sobre el primer qubit,
- ZZ mide Pauli-Z sobre ambos qubits.

Resumen Final

Ejemplo	Qubits	¿Qué se mide?	Observable Π_0
1	1 qubit	¿Está en $ 0\rangle$?	$\frac{1}{2}(I + Z)$
2	2 qubits	¿Está en $ 00\rangle$?	$\frac{1}{4}(II + IZ + ZI + ZZ)$

Conclusión: El observable cambia porque depende tanto del número de qubits como del estado específico que queremos detectar. Se usa el tensor producto para construir proyectores en sistemas de múltiples qubits.

IV. 3. COMPARACIÓN DE SECUENCIAS GENÉTICAS DE DIVERSOS ANIMALES

A. 3.1 Representación cuántica de secuencias genéticas reales

Para esta sección, se escogen cuatro secuencias genéticas reales pertenecientes a distintas especies: **Humano**, **Chimpancé**, **Perro** y **Oveja**. Estas cadenas están compuestas por los nucleótidos A, C, G y T, que forman el alfabeto genético de referencia.

```
#
nucleotides = {
    'A': 0,
    'C': 1,
    'G': 2,
    'T': 3
}
```

A diferencia del ejemplo anterior basado en cadenas binarias (“M” y “-”), cada carácter aquí puede tomar uno de **cuatro valores**, lo que requiere una representación más rica en términos cuánticos.

¿Cuántos qubits se requieren para representar estas secuencias?

Dado que se desea codificar cada carácter con una de 4 posibles bases (A, C, G, T), se necesitan $\log_2(4) = 2$ qubits por carácter para codificar su valor. Además, si la longitud de la cadena es L , es necesario direccionar cada una de las posiciones de los caracteres, lo que requiere $\log_2(L)$ qubits adicionales para direccionamiento.

Por tanto, el total de estados necesarios es $4 \cdot L$, lo que se traduce en $\log_2(4L) = 2 + \log_2(L)$ qubits.

Explicación de la codificación:

- Cada nucleótido se asigna a una posición dentro de un bloque de 4 amplitudes (representación binaria de dos qubits).
- Se define un vector de estado deseado de dimensión 2^n , donde $n = \lceil \log_2(4 \cdot L) \rceil$.
- A cada posición correspondiente a un carácter se le asigna una amplitud uniforme $\sqrt{1/L}$.

Distribución de qubits:

- **Para dirección:** $\log_2(L)$ qubits para identificar cada posición de la cadena.
- **Para contenido:** 2 qubits para codificar el tipo de nucleótido (A, C, G, T).

Representación como estado cuántico:

Dado el circuito generado, se obtiene un estado cuántico $|\psi\rangle$ que contiene una superposición de todos los estados base representando los caracteres de la cadena. Cada término representa un nucleótido codificado en la base computacional. Estos estados se usarán posteriormente para calcular la fidelidad entre especies.

B. 3.2 Implementación del cálculo de la fidelidad cuántica entre secuencias

Para comparar la similitud entre dos secuencias genéticas representadas como estados cuánticos, se utiliza el concepto de **fidelidad cuántica**. Dado un par de estados $|\psi_1\rangle$ y $|\psi_2\rangle$, la fidelidad se define como:

$$F = |\langle\psi_1|\psi_2\rangle|^2$$

Este valor está comprendido entre 0 y 1, donde 1 indica que los dos estados son idénticos.

Implementación en el código desarrollado:

Una vez generados los circuitos cuánticos para cada especie mediante la función `encode_bitstring`, se convierten en vectores de estado usando la clase `Statevector` de Qiskit. Luego, se compara cada par de vectores aplicando la definición algebraica de fidelidad.

```
1 # Inicializar variables para comparación
2 most_similar = None
3 most_similar_score = -1
4
5 # Comparar estados cuánticos (usando Statevector
  ↳ ya calculados en la celda anterior)
6 names = list(circuits.keys())
7 for i in range(len(names)):
8     for j in range(i + 1, len(names)):
9         name1, name2 = names[i], names[j]
10        state1 =
  ↳ Statevector.from_instruction(circuits[name1])
11        state2 =
  ↳ Statevector.from_instruction(circuits[name2])
12
13        fidelity =
  ↳ np.abs(np.dot(state1.data.conj(),
  ↳ state2.data))**2
14        print("La fidelidad entre {} y {} es:
  ↳ {:.3f}".format(name1, name2,
  ↳ fidelity))
15
16        if fidelity > most_similar_score:
17            most_similar, most_similar_score =
  ↳ (name1, name2), fidelity
18
19 print("\nEl par más similar es {} con fidelidad:
  ↳ {:.3f}".format(most_similar,
  ↳ most_similar_score))
```

Este fragmento realiza los siguientes pasos:

- Convierte el circuito cuántico en un vector de estado $|\psi\rangle$.
- Aplica el producto interno entre los vectores conjugados.
- Eleva el resultado al cuadrado para obtener la fidelidad F .

C. 3.3 Prueba de consistencia del programa cuántico calculando la similaridad de una de las secuencias genéticas consigo misma.

Como verificación del correcto funcionamiento del programa, se comparó una secuencia genética consigo misma. Para este fin, se utilizó la cadena genética del **Humano**.

```

1 State_Human_vector =
  ↳ Statevector.from_instruction(circuits['Human'])
2 fidelity =
  ↳ np.abs(np.dot(State_Human_vector.data.conj(),
  ↳ State_Human_vector.data))**2
3 print("La fidelidad para el caso de Humano con
  ↳ Humano es: {:.3f}".format(fidelity))

```

El resultado de esta comparación fue:

$$F(\text{Humano}, \text{Humano}) = 1.000$$

Este valor confirma que el programa calcula correctamente la fidelidad entre dos estados idénticos, resultando en una fidelidad perfecta.

A continuación, se modificó manualmente la cadena genética del humano en **cuatro nucleótidos** y se volvió a ejecutar el cálculo de fidelidad cuántica frente a la cadena original:

```

1 Altered_Human = ( "AGCCGGGCGCTGGCGCCACCGC" + ...
  ↳ ) # cadena modificada
2 n_Human = math.ceil(math.log2(4 *
  ↳ len(Altered_Human)))
3 qr_altered_Human = QuantumRegister(n_Human)
4 cr_altered_Human = ClassicalRegister(n_Human)
5
6 qc_altered_Human =
  ↳ encode_bitstring(Altered_Human,
  ↳ qr_altered_Human, cr_altered_Human,
  ↳ inverse=False)
7 State_altered_Human_vector =
  ↳ Statevector.from_instruction(qc_altered_Human)
8
9 fidelity =
10 np.abs(np.dot(State_altered_Human_vector.data.conj(),
  ↳ State_Human_vector.data))**2
11 print("La fidelidad para el caso de Humano con
  ↳ Humano alterado genéticamente es:
  ↳ {:.3f}".format(fidelity))

```

$$F(\text{Humano}, \text{Humano alterado}) = 0.987$$

Este resultado muestra que incluso una pequeña alteración en la cadena genética produce una disminución medible en la fidelidad cuántica. Por tanto, el modelo es sensible a cambios mínimos y puede ser útil para análisis de mutaciones genéticas.

D. 3.5 Comparación cuántica entre especies seleccionadas

Para determinar el grado de similitud genética entre distintas especies, se implementó una rutina en Python que calcula la **fidelidad cuántica** entre todos los pares posibles de secuencias genéticas codificadas. Las especies comparadas fueron: *Humano*, *Chimpancé*, *Perro* y *Oveja*.

Código utilizado para la comparación:

```

1 most_similar = None
2 most_similar_score = -1
3
4 names = list(circuits.keys())
5 for i in range(len(names)):
6     for j in range(i + 1, len(names)):
7         name1, name2 = names[i], names[j]
8         state1 =
9             ↳ Statevector.from_instruction(circuits[name1])
10        state2 =
11            ↳ Statevector.from_instruction(circuits[name2])
12
13        fidelity =
14            ↳ np.abs(np.dot(state1.data.conj(),
15            ↳ state2.data))**2
16        print("La fidelidad entre {} y {} es:
17            ↳ {:.3f}".format(name1, name2,
18            ↳ fidelity))
19
20        if fidelity > most_similar_score:
21            most_similar, most_similar_score =
22                ↳ (name1, name2), fidelity

```

Resultados obtenidos:

- $F(\text{Humano}, \text{Chimpancé}) = 0.957$
- $F(\text{Humano}, \text{Perro}) = 0.173$
- $F(\text{Humano}, \text{Oveja}) = 0.199$
- $F(\text{Chimpancé}, \text{Perro}) = 0.168$
- $F(\text{Chimpancé}, \text{Oveja}) = 0.202$
- $F(\text{Perro}, \text{Oveja}) = 0.146$

Par más similar encontrado:

(Humano, Chimpancé) con fidelidad cuántica $F = 0.957$

Este resultado refleja con precisión el conocimiento biológico previo: el humano y el chimpancé comparten un alto porcentaje de similitud genética, y esta característica se ve reflejada también desde la perspectiva de la computación cuántica mediante la comparación de sus estados $|\psi\rangle$. Las comparaciones con el perro y la oveja muestran valores mucho menores, indicando una menor coincidencia genética en esas secuencias.